

# Tool Calling in AI Agents

Consolidated Study Notes from Provided Notebooks

These notes consolidate the six uploaded notebooks:

- 001\_tools.ipynb
- 002\_tools\_call.ipynb
- 003\_tools\_call\_multiple\_turns.ipynb
- 004\_tool\_streaming.ipynb
- 005\_text\_editor\_tool.ipynb
- 006\_web\_search\_complete.ipynb

The document explains the theory, workflow, code logic, important snippets, output interpretation, and revision points for exams, interviews, and projects.

# Contents

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Concept Overview</b>                                       | <b>2</b>  |
| 1.1      | What is tool calling? . . . . .                               | 2         |
| 1.2      | Why is tool calling used? . . . . .                           | 2         |
| 1.3      | Where is it used in AI/ML and agentic AI? . . . . .           | 2         |
| 1.4      | Real-world examples . . . . .                                 | 2         |
| <b>2</b> | <b>Core Theory</b>                                            | <b>2</b>  |
| 2.1      | Main components . . . . .                                     | 2         |
| 2.2      | Tool-calling mental model . . . . .                           | 3         |
| 2.3      | Important terms . . . . .                                     | 3         |
| 2.3.1    | Tool schema . . . . .                                         | 3         |
| 2.3.2    | JSON input schema . . . . .                                   | 3         |
| 2.3.3    | Stop reason . . . . .                                         | 3         |
| 2.3.4    | Tool use ID . . . . .                                         | 3         |
| 2.4      | Key assumption . . . . .                                      | 4         |
| <b>3</b> | <b>Basic Tool Calling Workflow</b>                            | <b>4</b>  |
| 3.1      | Important point . . . . .                                     | 4         |
| <b>4</b> | <b>Code Explanation from the Provided Notebooks</b>           | <b>4</b>  |
| 4.1      | Notebook 001: Basic tools . . . . .                           | 4         |
| 4.1.1    | Helper functions . . . . .                                    | 4         |
| 4.1.2    | Current datetime tool . . . . .                               | 5         |
| 4.1.3    | Datetime tool schema . . . . .                                | 5         |
| 4.2      | Notebook 002: Tool call message handling . . . . .            | 5         |
| 4.3      | Notebook 003: Multiple-turn tool calling . . . . .            | 5         |
| 4.3.1    | Date calculation tool . . . . .                               | 6         |
| 4.3.2    | Reminder tool . . . . .                                       | 6         |
| 4.3.3    | Tool dispatcher . . . . .                                     | 6         |
| 4.3.4    | Run tools from model response . . . . .                       | 6         |
| 4.3.5    | Conversation loop . . . . .                                   | 7         |
| 4.4      | Notebook 004: Tool streaming . . . . .                        | 7         |
| 4.4.1    | Why streaming matters . . . . .                               | 8         |
| 4.4.2    | Save article schema . . . . .                                 | 8         |
| 4.4.3    | Streaming conversation idea . . . . .                         | 8         |
| 4.5      | Notebook 005: Text editor tool . . . . .                      | 8         |
| 4.5.1    | Purpose . . . . .                                             | 8         |
| 4.5.2    | Main commands . . . . .                                       | 9         |
| 4.5.3    | Path validation . . . . .                                     | 9         |
| 4.5.4    | Unique replacement safety . . . . .                           | 9         |
| 4.5.5    | Backups and undo . . . . .                                    | 9         |
| 4.5.6    | Special text-editor schema . . . . .                          | 9         |
| 4.6      | Notebook 006: Web search tool . . . . .                       | 9         |
| <b>5</b> | <b>Workflow / Algorithm Steps</b>                             | <b>10</b> |
| 5.1      | Algorithm 1: Single tool call . . . . .                       | 10        |
| 5.2      | Algorithm 2: Multi-turn tool calling . . . . .                | 10        |
| 5.3      | Algorithm 3: Multiple tool requests in one response . . . . . | 10        |

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| <b>6</b>  | <b>Important Code Snippets</b>                            | <b>10</b> |
| 6.1       | Snippet 1: Sending tools to the model . . . . .           | 10        |
| 6.2       | Snippet 2: Returning a tool result . . . . .              | 11        |
| 6.3       | Snippet 3: Dispatcher pattern . . . . .                   | 11        |
| 6.4       | Snippet 4: Safe text replacement . . . . .                | 11        |
| 6.5       | Snippet 5: Restricted web search . . . . .                | 11        |
| <b>7</b>  | <b>Mathematical / Formal View</b>                         | <b>12</b> |
| 7.1       | Tool selection . . . . .                                  | 12        |
| 7.2       | Tool execution . . . . .                                  | 12        |
| 7.3       | Common exam-style formula . . . . .                       | 12        |
| <b>8</b>  | <b>Output Interpretation</b>                              | <b>12</b> |
| 8.1       | Basic datetime example . . . . .                          | 12        |
| 8.2       | Multiple-turn reminder example . . . . .                  | 13        |
| 8.3       | Streaming output . . . . .                                | 13        |
| 8.4       | Text editor output . . . . .                              | 13        |
| 8.5       | Web search output . . . . .                               | 13        |
| <b>9</b>  | <b>Common Mistakes and Confusion Points</b>               | <b>13</b> |
| 9.1       | Mistake 1: Thinking the model runs the function . . . . . | 13        |
| 9.2       | Mistake 2: Forgetting to return tool results . . . . .    | 13        |
| 9.3       | Mistake 3: Losing the tool use ID . . . . .               | 13        |
| 9.4       | Mistake 4: Weak tool descriptions . . . . .               | 13        |
| 9.5       | Mistake 5: Executing arbitrary tool names . . . . .       | 13        |
| 9.6       | Mistake 6: No error handling . . . . .                    | 13        |
| 9.7       | Mistake 7: Unsafe file access . . . . .                   | 14        |
| <b>10</b> | <b>Best Practices</b>                                     | <b>14</b> |
| <b>11</b> | <b>Mini Project Architecture: Tool-Calling Agent</b>      | <b>14</b> |
| 11.1      | Recommended folder structure . . . . .                    | 14        |
| 11.2      | Execution flow . . . . .                                  | 14        |
| <b>12</b> | <b>Exam / Interview Questions</b>                         | <b>15</b> |
| <b>13</b> | <b>One-Shot Revision</b>                                  | <b>15</b> |
| 13.1      | Key points to remember . . . . .                          | 15        |
| 13.2      | Important code logic . . . . .                            | 15        |
| 13.3      | Most important exam answer . . . . .                      | 16        |

# 1 Concept Overview

## 1.1 What is tool calling?

**Tool calling** is the process where a language model decides that it needs an external function, API, or system action to answer a user's request. Instead of directly guessing the answer, the model outputs a structured tool request. Your application then executes the real tool and sends the result back to the model.

**Simple idea:** The model does not directly run Python, access files, set reminders, or search the web. It asks your program to do those actions using a tool call. Your program is the executor.

## 1.2 Why is tool calling used?

Tool calling is used because LLMs are strong at reasoning and language, but weak at reliable external actions. Tools allow the model to:

- get current data, such as time, dates, search results, or database records;
- perform deterministic calculations using code instead of guessing;
- interact with files, calendars, reminders, text editors, APIs, or databases;
- complete multi-step agent workflows, such as calculate a date first, then create a reminder;
- produce more reliable and auditable outputs.

## 1.3 Where is it used in AI/ML and agentic AI?

Tool calling appears in agentic AI systems, RAG pipelines, chatbots, workflow automation, coding agents, data-analysis assistants, web-browsing assistants, and compliance auditors. In an AI agent, the model usually acts as the **planner**, while tools act as the **workers**.

## 1.4 Real-world examples

- A customer support bot calls a ticketing API to create a support ticket.
- A coding agent calls a text-editor tool to view, replace, insert, or undo code edits.
- A medical information assistant calls a web-search tool restricted to reliable medical domains.
- A reminder assistant first calls a date-calculation tool, then calls a reminder-setting tool.
- A data-science assistant calls Python to clean a dataset and return evaluation metrics.

# 2 Core Theory

## 2.1 Main components

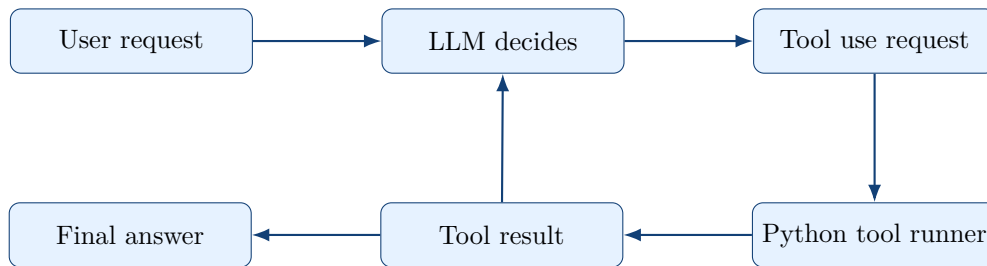
| Component             | Meaning                                                                                                              |
|-----------------------|----------------------------------------------------------------------------------------------------------------------|
| User message          | The user's natural-language request. Example: <b>What is the exact time?</b>                                         |
| Assistant message     | The model's response. It may contain normal text or one or more <code>tool_use</code> blocks.                        |
| Tool/function         | Real executable code outside the model, such as <code>get_current_datetime()</code> or <code>set_reminder()</code> . |
| Tool schema           | A structured description of the tool: name, description, parameters, parameter types, and required fields.           |
| <code>tool_use</code> | The model's structured request to call a tool with arguments.                                                        |

|                          |                                                                                                                        |
|--------------------------|------------------------------------------------------------------------------------------------------------------------|
| Tool runner              | Your Python code that maps the requested tool name to the actual function.                                             |
| <code>tool_result</code> | The returned result sent back to the model, linked to the original tool request ID.                                    |
| Conversation loop        | The repeated process of asking the model, running tools, returning results, and continuing until no tool is requested. |

---

## 2.2 Tool-calling mental model

Think of the model as a student in an exam who can ask a calculator to compute something. The student decides *what* to calculate, but the calculator gives the exact result. Similarly, the LLM decides which tool to call and what arguments to pass, while the application executes the tool.



## 2.3 Important terms

### 2.3.1 Tool schema

A tool schema is like the contract between the LLM and your program. It tells the model:

- the tool name;
- what the tool does;
- what inputs are allowed;
- which inputs are required;
- what each input means.

A good schema improves tool accuracy. If the description is vague, the model may call the wrong tool or pass wrong arguments.

### 2.3.2 JSON input schema

Most tool definitions use a JSON Schema-like structure. Example fields:

- `type`: usually `object`;
- `properties`: dictionary of accepted arguments;
- `required`: list of mandatory parameters;
- `description`: natural-language guidance for the model.

### 2.3.3 Stop reason

In the notebooks, the model response has a stop reason. A common important value is `tool_use`. That means the model is not done yet; it wants the application to run one or more tools.

### 2.3.4 Tool use ID

Every tool request has an ID. When sending a result back, the result must include the matching `tool_use_id`. This links the output to the exact request.

## 2.4 Key assumption

The LLM is not trusted to execute actions directly. The application must validate tool names, validate arguments, run only allowed tools, handle errors, and return safe results.

## 3 Basic Tool Calling Workflow

The basic tool-calling workflow from 001\_tools.ipynb and 002\_tools\_call.ipynb is:

1. Define a real Python function, such as `get_current_datetime()`.
2. Create a schema that describes the function to the model.
3. Send the user's message and the tool schema to the model.
4. The model returns a `tool_use` block if it needs the tool.
5. Extract the tool name and arguments from the response.
6. Execute the matching Python function locally.
7. Add a `tool_result` message to the conversation.
8. Call the model again so it can convert the tool output into a user-friendly answer.

### 3.1 Important point

The model's first response may not be the final answer. It may only be a request like:

```
ToolUseBlock(  
    name='get_current_datetime',  
    input={'date_format': '%H:%M:%S'}  
)
```

After your code returns the result, such as 13:56:07, the model can respond: *The exact time is 13:56:07.*

## 4 Code Explanation from the Provided Notebooks

### 4.1 Notebook 001: Basic tools

This notebook introduces the minimum manual flow for tool calling.

#### 4.1.1 Helper functions

The notebook defines helpers for adding user and assistant messages to the conversation list.

```
def add_user_message(messages, text):  
    user_message = {"role": "user", "content": text}  
    messages.append(user_message)  
  
def add_assistant_message(messages, text):  
    assistant_message = {"role": "assistant", "content": text}  
    messages.append(assistant_message)
```

#### Explanation:

- `messages` stores the full conversation history.
- Each message has a `role`: either `user` or `assistant`.
- The model needs the full history to understand previous tool calls and results.

### 4.1.2 Current datetime tool

The function below returns the current date/time in a requested format.

```
def get_current_datetime(date_format="%Y-%m-%d %H:%M:%S"):
    if not date_format:
        raise ValueError("date_format cannot be empty")
    return datetime.now().strftime(date_format)
```

**Why it is needed:** A model should not guess the current time. The actual system time is fetched by code.

### 4.1.3 Datetime tool schema

```
get_current_datetime_schema = ToolParam({
    "name": "get_current_datetime",
    "description": "Returns the current date and time formatted according to a Python
    strftime date format string.",
    "input_schema": {
        "type": "object",
        "properties": {
            "date_format": {
                "type": "string",
                "description": "Python strftime format, such as %H:%M:%S"
            }
        },
        "required": []
    }
})
```

#### Explanation:

- name must match the tool name the runner expects.
- description guides the model on when to use the tool.
- input\_schema tells the model what arguments it may pass.
- date\_format lets the model request formats like %H:%M:%S.

## 4.2 Notebook 002: Tool call message handling

This notebook repeats the same idea but improves the helper functions to handle model `Message` objects and content blocks.

```
def add_user_message(messages, message):
    user_message = {
        "role": "user",
        "content": message.content if isinstance(message, Message) else message,
    }
    messages.append(user_message)
```

**Why this matters:** After a model response, the content is not always plain text. It may be a structured list containing text blocks and tool-use blocks. The helper handles both raw strings and message objects.

## 4.3 Notebook 003: Multiple-turn tool calling

This notebook shows a more agent-like workflow. The user asks:

Set a reminder for my doctor's appointment. It is 177 days after Jan 1st, 2050.

The model cannot set the reminder immediately because it first needs to calculate the date. So it does two tool calls across the same conversation:

1. Call `add_duration_to_datetime` with `datetime_str='2050-01-01'` and `duration=177`.
2. Use the returned date to call `set_reminder`.

#### 4.3.1 Date calculation tool

```
def add_duration_to_datetime(datetime_str, duration=0, unit="days", input_format="%Y-%m-%d"):
    date = datetime.strptime(datetime_str, input_format)

    if unit == "days":
        new_date = date + timedelta(days=duration)
    elif unit == "weeks":
        new_date = date + timedelta(weeks=duration)
    elif unit == "months":
        # special month handling because months have different lengths
        ...
    else:
        raise ValueError(f"Unsupported time unit: {unit}")

    return new_date.strftime("%A, %B %d, %Y %I:%M:%S %p")
```

**Exam-friendly intuition:** This tool converts a string into a `datetime` object, adds a duration, then converts it back into a readable string.

#### 4.3.2 Reminder tool

```
def set_reminder(content, timestamp):
    print(f"Setting the following reminder for {timestamp}:\n{content}")
```

In a real app, this would call a reminder service or database. In the notebook, it only prints the action, which is useful for demonstration.

#### 4.3.3 Tool dispatcher

```
def run_tool(tool_name, tool_input):
    if tool_name == "get_current_datetime":
        return get_current_datetime(**tool_input)
    elif tool_name == "add_duration_to_datetime":
        return add_duration_to_datetime(**tool_input)
    elif tool_name == "set_reminder":
        return set_reminder(**tool_input)
```

##### Explanation:

- The model outputs only a tool name and input.
- Your application maps that name to a real Python function.
- `**tool_input` unpacks the dictionary into keyword arguments.

#### 4.3.4 Run tools from model response

```
def run_tools(message):
    tool_requests = [
        block for block in message.content if block.type == "tool_use"
    ]
    tool_result_blocks = []

    for tool_request in tool_requests:
        try:
            tool_output = run_tool(tool_request.name, tool_request.input)
```



```

        tool_result_blocks.append({
            "type": "tool_result",
            "tool_use_id": tool_request.id,
            "content": str(tool_output),
            "is_error": False,
        })
    except Exception as e:
        tool_result_blocks.append({
            "type": "tool_result",
            "tool_use_id": tool_request.id,
            "content": str(e),
            "is_error": True,
        })

    return tool_result_blocks

```

#### Why it is important:

- It can handle multiple tool calls in one assistant response.
- It catches errors and returns them to the model in a structured way.
- It preserves the `tool_use_id`, which is required to connect result to request.

#### 4.3.5 Conversation loop

```

def run_conversation(messages):
    while True:
        response = chat(
            messages,
            tools=[get_current_datetime_schema,
                  add_duration_to_datetime_schema,
                  set_reminder_schema]
        )

        add_assistant_message(messages, response)
        print(text_from_message(response))

        if response.stop_reason != "tool_use":
            break

        tool_results = run_tools(response)
        add_user_message(messages, tool_results)

    return messages

```

#### The loop logic:

1. Ask the model what to do next.
2. If it answers normally, stop.
3. If it requests a tool, run the tool.
4. Add the tool result as a user message.
5. Repeat.

### 4.4 Notebook 004: Tool streaming

This notebook shows how to stream model output while the tool call is being generated.

#### 4.4.1 Why streaming matters

Without streaming, you wait until the full response is generated. With streaming, you can display:

- normal text as it arrives;
- when a tool call starts;
- partial JSON tool input chunks;
- the final tool call result.

#### 4.4.2 Save article schema

```
save_article_schema = ToolParam({
    "name": "save_article",
    "description": "Saves a scholarly journal article",
    "input_schema": {
        "type": "object",
        "properties": {
            "abstract": {"type": "string"},
            "meta": {
                "type": "object",
                "properties": {
                    "word_count": {"type": "integer"},
                    "review": {"type": "string"}
                },
                "required": ["word_count", "review"]
            },
        },
        "required": ["abstract", "meta"]
    }
})
```

**Important observation:** This schema has a nested object, `meta`. Tool schemas can represent complex structured inputs, not just simple strings.

#### 4.4.3 Streaming conversation idea

```
with chat_stream(messages, tools=tools, betas=["fine-grained-tool-streaming-2025-05-14"]):
    for chunk in stream:
        if chunk.type == "text":
            print(chunk.text, end="")
        if chunk.type == "content_block_start":
            if chunk.content_block.type == "tool_use":
                print(f"Tool Call: {chunk.content_block.name}")
        if chunk.type == "input_json":
            print(chunk.partial_json, end="")
```

**Study point:** Fine-grained tool streaming is useful when tool arguments are long, such as article metadata, generated JSON, or code edits.

### 4.5 Notebook 005: Text editor tool

This notebook builds a local text editor tool with commands such as view, replace, create, insert, and undo.

#### 4.5.1 Purpose

A text-editor tool lets a coding agent inspect and modify files. The model decides which command to use, but the application controls the file system.

### 4.5.2 Main commands

| Command     | Purpose                                                               |
|-------------|-----------------------------------------------------------------------|
| view        | Read a file or list a directory. Can include line numbers and ranges. |
| str_replace | Replace exactly one occurrence of a string with new text.             |
| create      | Create a new file with given text.                                    |
| insert      | Insert text after a specified line number.                            |
| undo_edit   | Restore the latest backup for a file.                                 |

### 4.5.3 Path validation

```
def _validate_path(self, file_path: str) -> str:
    abs_path = os.path.normpath(os.path.join(self.base_dir, file_path))
    if not abs_path.startswith(self.base_dir):
        raise ValueError("Access denied: Path is outside the allowed directory")
    return abs_path
```

**Why it is important:** Tool-calling agents must not freely access the entire file system. Path validation prevents directory traversal attacks such as `../../secret.txt`.

### 4.5.4 Unique replacement safety

```
match_count = self._count_matches(content, old_str)

if match_count == 0:
    raise ValueError("No match found for replacement.")
elif match_count > 1:
    raise ValueError("Found multiple matches. Provide more context.")
```

**Why it is important:** Code-editing tools should avoid ambiguous edits. If the old string occurs more than once, the tool refuses to modify the file until the model gives more specific context.

### 4.5.5 Backups and undo

Before editing, the tool creates a backup. If something goes wrong, `undo_edit` can restore the latest backup. This is a key safety feature for coding agents.

### 4.5.6 Special text-editor schema

```
def get_text_edit_schema(model):
    return {
        "type": "text_editor_20250728",
        "name": "str_replace_based_edit_tool",
    }
```

Unlike custom JSON schemas, this uses a built-in text editor tool type. The model already understands the supported commands.

## 4.6 Notebook 006: Web search tool

This notebook demonstrates a built-in web-search tool.

```
web_search_schema = {
    "type": "web_search_20250305",
    "name": "web_search",
    "max_uses": 5,
    "allowed_domains": ["nih.gov"],
}
```

**Explanation:**

- `type` selects a built-in web search capability.
- `name` is the tool name the model sees.
- `max_uses` limits how many searches the model can perform.
- `allowed_domains` restricts the search to trusted sites. Here, it is limited to `nih.gov`.

## 5 Workflow / Algorithm Steps

### 5.1 Algorithm 1: Single tool call

1. Receive user query.
2. Send query plus available tool schema to the model.
3. Check whether the model response contains `tool_use`.
4. If no tool call exists, return the answer.
5. If a tool call exists, validate the tool name.
6. Validate and execute the arguments.
7. Return a `tool_result` message.
8. Ask the model again for the final response.

### 5.2 Algorithm 2: Multi-turn tool calling

1. Start with the user request.
2. Let the model plan the next action.
3. If the next action is a tool, run the tool.
4. Send the result back to the model.
5. The model may use the result to call another tool.
6. Repeat until `stop_reason != "tool_use"`.

### 5.3 Algorithm 3: Multiple tool requests in one response

1. Search through the assistant content blocks.
2. Select blocks where `block.type == "tool_use"`.
3. For each tool request, call the dispatcher.
4. Convert each output into a `tool_result` block.
5. Return the list of result blocks together.

## 6 Important Code Snippets

### 6.1 Snippet 1: Sending tools to the model

```
response = client.messages.create(  
    model=model,  
    max_tokens=1000,  
    messages=messages,  
    tools=[get_current_datetime_schema]  
)
```

### Line-by-line:

- `model=model`: selects the model.
- `max_tokens=1000`: limits response length.
- `messages=messages`: passes conversation history.
- `tools=[...]`: tells the model which tools are available.

## 6.2 Snippet 2: Returning a tool result

```
messages.append({
    "role": "user",
    "content": [{
        "type": "tool_result",
        "tool_use_id": response.content[0].id,
        "content": result,
        "is_error": False
    }]
})
```

**Role in workflow:** This message gives the tool output back to the model. It is added with role `user` because it represents external information returned to the assistant.

## 6.3 Snippet 3: Dispatcher pattern

```
def run_tool(tool_name, tool_input):
    if tool_name == "get_current_datetime":
        return get_current_datetime(**tool_input)
    elif tool_name == "add_duration_to_datetime":
        return add_duration_to_datetime(**tool_input)
    elif tool_name == "set_reminder":
        return set_reminder(**tool_input)
    else:
        raise ValueError(f"Unknown tool: {tool_name}")
```

**Best practice:** Always include an `else` branch for unknown tools. Never execute arbitrary tool names using `eval`.

## 6.4 Snippet 4: Safe text replacement

```
if match_count == 0:
    raise ValueError("No match found")
elif match_count > 1:
    raise ValueError("Multiple matches found")

self._backup_file(abs_path)
new_content = content.replace(old_str, new_str)
```

**Role in workflow:** This prevents accidental edits and creates a backup before modifying a file.

## 6.5 Snippet 5: Restricted web search

```
web_search_schema = {
    "type": "web_search_20250305",
    "name": "web_search",
    "max_uses": 5,
    "allowed_domains": ["nih.gov"],
}
```

**Role in workflow:** This restricts the model to a specific trusted domain, which is useful for high-stakes topics.

## 7 Mathematical / Formal View

Tool calling does not have heavy mathematics, but the workflow can be represented formally.

### 7.1 Tool selection

Let:

- $q$  be the user query,
- $H$  be the conversation history,
- $T = \{t_1, t_2, \dots, t_n\}$  be the available tools,
- $s_i$  be the schema for tool  $t_i$ .

The model chooses either a final answer  $a$  or a tool call:

$$\text{LLM}(q, H, S) \rightarrow a \quad \text{or} \quad (t_i, x_i)$$

where  $S = \{s_1, s_2, \dots, s_n\}$  and  $x_i$  is the JSON input for tool  $t_i$ .

### 7.2 Tool execution

The application executes:

$$y_i = t_i(x_i)$$

where  $y_i$  is the tool output. Then the conversation is updated:

$$H' = H \cup \{\text{tool\_result}(id_i, y_i)\}$$

Finally, the model uses the updated history:

$$\text{LLM}(q, H', S) \rightarrow \text{final answer}$$

### 7.3 Common exam-style formula

For multi-step agents:

$$H_{k+1} = H_k \cup \{\text{assistant tool\_use}_k, \text{user tool\_result}_k\}$$

The loop stops when:

$$\text{stop\_reason} \neq \text{"tool\_use"}$$

## 8 Output Interpretation

### 8.1 Basic datetime example

In the notebooks, the user asks for exact time in HH:MM:SS format. The model responds with a tool request:

```
input={'date_format': '%H:%M:%S'}, name='get_current_datetime'
```

The Python function returns a value such as:

```
13:56:07
```

Then the model converts it into a final natural-language answer.

## 8.2 Multiple-turn reminder example

The output shows that the model first explains its plan, then calls the date tool, then calls the reminder tool. The important thing is that the model used the first tool output as input for the next action.

**Key interpretation:** A multi-turn tool workflow is not just one function call. It is a reasoning loop where every tool result becomes new context for the next model decision.

## 8.3 Streaming output

In the streaming notebook, output appears gradually. The model first prints text, then starts a tool call, then streams the JSON input. This is useful for debugging because you can see what arguments the model is constructing.

## 8.4 Text editor output

The text editor tool's `view` command returns line-numbered file content. The model then summarizes the file. In the notebook, `main.py` contains a simple `greeting()` function that prints `Hi there`.

## 8.5 Web search output

The web search notebook asks about exercises for gaining leg muscle. The important part is not the exercise answer itself, but the tool setup: the search is restricted using `max_uses` and `allowed_domains`. This demonstrates controlled retrieval.

# 9 Common Mistakes and Confusion Points

## 9.1 Mistake 1: Thinking the model runs the function

The model does not run the function. It only requests the function call. Your application runs it.

## 9.2 Mistake 2: Forgetting to return tool results

If you run the tool but do not add a `tool_result` message, the model cannot see the result.

## 9.3 Mistake 3: Losing the tool use ID

The result must include the matching `tool_use_id`. Otherwise, the model may not know which result belongs to which call.

## 9.4 Mistake 4: Weak tool descriptions

Bad descriptions lead to bad tool usage. Good descriptions explain when to use the tool and what each argument means.

## 9.5 Mistake 5: Executing arbitrary tool names

Never do this:

```
return globals()[tool_name](**tool_input)
```

Use a safe dispatcher instead.

## 9.6 Mistake 6: No error handling

Tools can fail. Always catch exceptions and return structured errors using `is_error=True`.

## 9.7 Mistake 7: Unsafe file access

A file tool must validate paths and restrict the base directory. Otherwise, the agent may access unintended files.

## 10 Best Practices

1. **Use precise schema descriptions.** Explain exactly when the tool should be called.
2. **Keep tools small and focused.** One tool should do one clear job.
3. **Validate inputs.** Check required fields, data types, allowed values, and paths.
4. **Use a dispatcher.** Map tool names explicitly to functions.
5. **Return structured results.** Include success or error states clearly.
6. **Limit high-risk tools.** File tools, web tools, shell tools, and email tools need strict permissions.
7. **Use domain restrictions for search.** Restrict web search when reliability matters.
8. **Log tool calls.** This helps debugging and auditing.
9. **Support undo for edit tools.** Always create backups before modifying files.
10. **Loop until done.** Continue calling the model until it no longer asks for tools.

## 11 Mini Project Architecture: Tool-Calling Agent

### 11.1 Recommended folder structure

```
agent_project/  
  app.py  
  tools/  
    datetime_tools.py  
    reminder_tools.py  
    file_tools.py  
    web_tools.py  
  schemas/  
    datetime_schema.py  
    reminder_schema.py  
  runner.py  
  logs/
```

### 11.2 Execution flow

1. Load tool schemas.
2. Start messages list.
3. Add user message.
4. Call model with tool schemas.
5. Parse `tool_use` blocks.
6. Run the matching tools.
7. Append `tool_result` blocks.
8. Repeat until final response.



## 12 Exam / Interview Questions

1. What is tool calling in LLM applications?
2. Why do we need a tool schema?
3. What is the difference between `tool_use` and `tool_result`?
4. Why must the `tool_use_id` be returned with the result?
5. Explain the dispatcher pattern for tool execution.
6. How does multi-turn tool calling work?
7. What is the purpose of `stop_reason == "tool_use"`?
8. Why should tools validate inputs?
9. What safety checks are needed for a file-editing tool?
10. How does web-search restriction improve reliability?
11. Why is streaming useful for tool calls?
12. What can go wrong if a replacement string appears multiple times in a file-editing tool?

## 13 One-Shot Revision

**Tool calling in one paragraph:** The user asks a question. The model receives the question plus tool schemas. If the model needs external help, it outputs a structured `tool_use` request with a tool name and JSON arguments. The application validates and executes the matching function, then sends a `tool_result` back using the same tool-use ID. The model reads the result and either answers the user or asks for another tool. This loop continues until the model stops requesting tools.

### 13.1 Key points to remember

- Tool schema = contract between model and application.
- Model decides the tool and arguments; application executes.
- `tool_use` is the request; `tool_result` is the response.
- Multi-turn tools are needed when one tool result feeds another tool call.
- Text-editor tools need path validation, backups, unique replacements, and undo.
- Web-search tools should be restricted when reliability matters.
- Always handle errors and never execute arbitrary code from the model.

### 13.2 Important code logic

```
while True:
    response = chat(messages, tools=tools)
    add_assistant_message(messages, response)

    if response.stop_reason != "tool_use":
        break

    tool_results = run_tools(response)
    add_user_message(messages, tool_results)
```

### 13.3 Most important exam answer

Tool calling allows an LLM to interact with external functions through structured requests. The model does not execute the tool itself. It emits a `tool_use` block, the application runs the corresponding function, and the result is returned as a `tool_result` block. This enables reliable external actions such as date calculation, file editing, reminder creation, and web search.